```
# If the array we are trying to sort has fewer than 64 elements in it,
#Timsort will execute an insertion sort.

#If the list is larger than 64 elements than the algorithm will make a first
pass through the list looking for parts that are strictly increasing or
decreasing.
#If the part is decreasing, it will reverse that part.

#The minrun is a size determined based on the size of array, so that most runs
in a random array are, or become minrun, in length.
#The algorithm chooses minrun from the range 32 to 64 inclusive. It chooses
minrun such that the length of the original array, when divided by minrun, is
equal to or slightly less than a power of two(more efficient).
#If the length of the run is less than minrun, you calculate the length of
that run away from minrun. Using this new number, you grab that many items
ahead of the run and perform an insertion sort to create a new run.

# Timsort now performs mergesort to merge the runs together. However, Timsort
makes sure to maintain stability and merge balance whilst merge sorting.

# To maintain stability we should not exchange 2 numbers of equal value. This
not only keeps their original positions in the list but enables the algorithm
to be faster. We will shortly discuss the merge balance.
# As Timsort finds runs, it adds them to a stack
# Timsort tries to balance two competing needs when mergesort runs. On one
hand, we would like to delay merging as long as possible in order to exploit
patterns that may come up later. But we would like even more to do the merging
as soon as possible to exploit the run that the run just found is still high
in the memory hierarchy. We also can't delay merging "too long" because it
consumes memory to remember the runs that are still unmerged, and the stack
has a fixed size.
# To make sure we have this compromise,
# Timsort keeps track of the three most recent items on the stack and creates
two laws that must hold true of those items:
# A > B + C
# B > C
# Where A, B and C are the three most recent items on the stack.

# Timsort assumes that if a lot of run A's values are lower than run B's
values, then it is likely that A will continue to have smaller values than B.
# Timsort will then enter galloping mode. Instead of checking A[0] and B[0]
against each other, Timsort performs a binary search for the appropriate
position of b[0] in a[0]. This way, Timsort can move a whole section of A into
place. Then Timsort searches for the appropriate location of A[0] in B.
Timsort will then move a whole section of B can at once, and into place.
# It turns out, this operation is not worth it if the appropriate location for
B[0] is very close to the beginning of A (or vice versa). so gallop mode
quickly exits if it isn't paying off. Additionally, Timsort takes note and
```

```
makes it harder to enter gallop mode later by increasing the number of
consecutive A-only or B-only wins required to enter. If gallop mode is paying
off, Timsort makes it easier to reenter.


def binary_search(the_array, item, start, end):
    if start == end:
        if the_array[start] > item:
            return start
        else:
            return start + 1
    if start > end:
        return start

    mid = round((start + end)/ 2)

    if the_array[mid] < item:
        return binary_search(the_array, item, mid + 1, end)

    elif the_array[mid] > item:
        return binary_search(the_array, item, start, mid - 1)

    else:
        return mid

"""
Insertion sort that timsort uses if the array size is small or if
the size of the "run" is small
"""
def insertion_sort(the_array):
    l = len(the_array)
    for index in range(1, l):
        value = the_array[index]
        pos = binary_search(the_array, value, 0, index - 1)
        the_array = the_array[:pos] + [value] + the_array[pos:index] +
the_array[index+1:]
    return the_array

def merge(left, right):
    """Takes two sorted lists and returns a single sorted list by comparing
the
    elements one at a time.
    [1, 2, 3, 4, 5, 6]
    """
    if not left:
        return right
    if not right:
        return left
    if left[0] < right[0]:
```

```
        return [left[0]] + merge(left[1:], right)
    return [right[0]] + merge(left, right[1:])

def timsort(the_array):
    runs, sorted_runs = [], []
    length = len(the_array)
    new_run = [the_array[0]]

    # for every i in the range of 1 to length of array
    for i in range(1, length):
        # if i is at the end of the list
        if i == length - 1:
            new_run.append(the_array[i])
            runs.append(new_run)
            break
        # if the i'th element of the array is less than the one before it
        if the_array[i] < the_array[i-1]:
            # if new_run is set to None (NULL)
            if not new_run:
                runs.append([the_array[i]])
                new_run.append(the_array[i])
            else:
                runs.append(new_run)
                new_run = []
        # else if its equal to or more than
        else:
            new_run.append(the_array[i])

    # for every item in runs, append it using insertion sort
    for item in runs:
        sorted_runs.append(insertion_sort(item))

    # for every run in sorted_runs, merge them
    sorted_array = []
    for run in sorted_runs:
        sorted_array = merge(sorted_array, run)

    print(sorted_array)

timsort([2, 3, 1, 5, 6, 7])
```

```python
# Python3 program to perform basic timSort
MIN_MERGE = 32


def calcMinRun(n):
    """Returns the minimum length of a
    run from 23 - 64 so that
    the len(array)/minrun is less than or
    equal to a power of 2.

    e.g. 1=>1, ..., 63=>63, 64=>32, 65=>33,
    ..., 127=>64, 128=>32, ...
    """
    r = 0
    while n >= MIN_MERGE:
        r |= n & 1
        n >>= 1
    return n + r


# This function sorts array from left index to
# to right index which is of size atmost RUN
def insertionSort(arr, left, right):
    for i in range(left + 1, right + 1):
        j = i
        while j > left and arr[j] < arr[j - 1]:
            arr[j], arr[j - 1] = arr[j - 1], arr[j]
            j -= 1


# Merge function merges the sorted runs
def merge(arr, l, m, r):

    # original array is broken in two parts
    # left and right array
    len1, len2 = m - l + 1, r - m
    left, right = [], []
    for i in range(0, len1):
        left.append(arr[l + i])
    for i in range(0, len2):
```

```
        right.append(arr[m + 1 + i])

    i, j, k = 0, 0, l

    # after comparing, we merge those two array
    # in larger sub array
    while i < len1 and j < len2:
        if left[i] <= right[j]:
            arr[k] = left[i]
            i += 1

        else:
            arr[k] = right[j]
            j += 1

        k += 1

    # Copy remaining elements of left, if any
    while i < len1:
        arr[k] = left[i]
        k += 1
        i += 1

    # Copy remaining element of right, if any
    while j < len2:
        arr[k] = right[j]
        k += 1
        j += 1


# Iterative Timsort function to sort the
# array[0...n-1] (similar to merge sort)
def timSort(arr):
    n = len(arr)
    minRun = calcMinRun(n)

    # Sort individual subarrays of size RUN
    for start in range(0, n, minRun):
        end = min(start + minRun - 1, n - 1)
        insertionSort(arr, start, end)

    # Start merging from size RUN (or 32). It will merge
    # to form size 64, then 128, 256 and so on ....
    size = minRun
    while size < n:

        # Pick starting point of left sub array. We
        # are going to merge arr[left..left+size-1]
        # and arr[left+size, left+2*size-1]
```

```
        # After every merge, we increase left by 2*size
        for left in range(0, n, 2 * size):

            # Find ending point of left sub array
            # mid+1 is starting point of right sub array
            mid = min(n - 1, left + size - 1)
            right = min((left + 2 * size - 1), (n - 1))

            # Merge sub array arr[left.....mid] &
            # arr[mid+1....right]
            if mid < right:
                merge(arr, left, mid, right)

        size = 2 * size


# Driver program to test above function
if __name__ == "__main__":

    arr = [-2, 7, 15, -14, 0, 15, 0,
           7, -7, -4, -13, 5, 8, -14, 12]

    print("Given Array is")
    print(arr)

    # Function Call
    timSort(arr)

    print("After Sorting Array is")
    print(arr)
```